

Taller Práctico: Asincronía y Callbacks en JavaScript

Objetivo

Implementar y analizar soluciones utilizando callbacks y tareas asíncronas en JavaScript para comprender cómo funciona el comportamiento no bloqueante del lenguaje.

Parte 1. Callbacks Básicos

Un callback es una función que se envía como argumento a otra función para ser ejecutada posteriormente.

Analiza el siguiente código y completa las secciones marcadas con TODO.

```
function realizarOperacion(num1, num2, operacionCallback)
```

```
    {  
        console.log(`Operación con: ${num1} y ${num2}`);  
        operacionCallback(num1, num2);  
    }  
}
```

```
function sumar(a, b) {  
    console.log(`Resultado Suma: ${a + b}`);  
}
```

```
// TODO  
function restar(a, b) {  
  
}
```

```
// TODO  
function multiplicar(a, b) {  
  
}
```

```
realizarOperacion(10, 5, sumar);
```

```
// TODO  
// Implementar las tres operaciones.
```

Actividades

1. Implementa el callback **restar**.
2. Implementa el callback **multiplicar**.
3. Ejecuta las tres operaciones.
4. Captura la salida obtenida en consola.

```
function sumar(a, b) {  
  console.log('Resultado Suma: ${a + b}'); 'Resultado Suma: 15'  
}  
  
function restar(a, b) {  
  console.log('Resultado Resta: ${a - b}'); 'Resultado Resta: 5'  
}  
  
function multiplicar(a, b) {  
  console.log('Resultado Multiplicación: ${a * b}'); 'Resultado Multiplicación: 50'  
}  
  
realizarOperacion(10, 5, sumar);  
realizarOperacion(10, 5, restar);  
realizarOperacion(10, 5, multiplicar);
```

Preguntas de reflexión

1. ¿Qué ventaja tiene enviar una función como parámetro?
La ventaja es que permite reutilizar código y hacer que una función pueda realizar diferentes tareas dependiendo del callback que reciba.
2. ¿Qué ocurre si enviamos una función diferente como callback?
La función principal ejecutará la nueva función enviada, por lo que el comportamiento cambiará sin necesidad de modificar el código principal.
3. ¿La función principal necesita conocer qué operación matemática se ejecutará?
No. La función principal solo recibe y ejecuta el callback, sin importar cuál sea la operación.

Parte 2. Comprendiendo la Asincronía

Observa cuidadosamente el siguiente programa.

```
function tareaNoBloqueante(callback) {  
  console.log("Iniciando tarea no bloqueante...");  
  
  setTimeout(function() {  
    console.log("Tarea completada.");  
    callback();  
  }, 2000);  
}  
  
console.log("Inicio del programa.");  
  
tareaNoBloqueante(function() {  
  console.log("Continuando después de la tarea.");  
});  
  
console.log("Fin del programa.");
```

Actividad

Antes de ejecutar el programa:

1. Escribe el orden que crees que aparecerá en la consola.
2. Ejecuta el código.
3. Registra el resultado real.

Análisis

Responde:

1. ¿Coincidió tu predicción con el resultado?
Sí, la predicción coincidió con el resultado obtenido al ejecutar el programa.
2. ¿Por qué el mensaje "Fin del programa" aparece antes que "Tarea completada"?
Porque `setTimeout()` ejecuta la tarea de forma asíncrona. Mientras espera los 2 segundos, JavaScript continúa ejecutando el resto del código.
3. ¿Qué papel cumple `setTimeout()` en este comportamiento?
este programa la ejecución de una función después de un tiempo determinado sin detener la ejecución del programa.

Parte 3. Simulación de Procesos Reales

Debes simular tres tareas comunes de una aplicación moderna.

Código Base

```
function solicitarJSON(callback) {  
  
}  
  
function reproducirAudio(callback) {  
  
}  
  
function leerSensor(callback) {  
  
}  
  
console.log("--- Iniciando procesos asíncronos ---");  
  
solicitarJSON(() => console.log("Archivo JSON recibido."));  
reproducirAudio(() => console.log("Audio finalizado."));  
leerSensor(() => console.log("Datos del sensor listos."));
```

Requisitos

Implementa cada función utilizando `setTimeout()`.

Función	Tiempo
solicitarJSON	3000 ms
reproducirAudio	1000 ms

```
function solicitarJSON(callback) {  
  setTimeout(() => {  
    callback();  
  }, 3000);  
}  
  
function reproducirAudio(callback) {  
  setTimeout(() => {  
    callback();  
  }, 1000);  
}  
  
function leerSensor(callback) {  
  setTimeout(() => {  
    callback();  
  }, 500);  
}
```

leerSensor 500 ms

Actividades

1. Implementa las tres funciones.
2. Ejecuta el programa.
3. Registra el orden real de finalización.

Resultado

Orden	Tarea
-------	-------

1	
---	--

2	
---	--

3	
---	--

Preguntas

1. ¿Cuál terminó primero?
leerSensor()
2. ¿Cuál terminó último?
solicitarJSON()
3. ¿Por qué el orden de finalización es diferente al orden de ejecución?
Porque cada tarea tiene un tiempo de espera diferente. Las tareas terminan según el tiempo que tardan y no según el orden en que fueron llamadas.

Parte 4. Diseñando tu Propia Operación Asíncrona

Reto

Crea una función llamada:

simularDescarga(nombreArchivo, callback)

La función deberá:

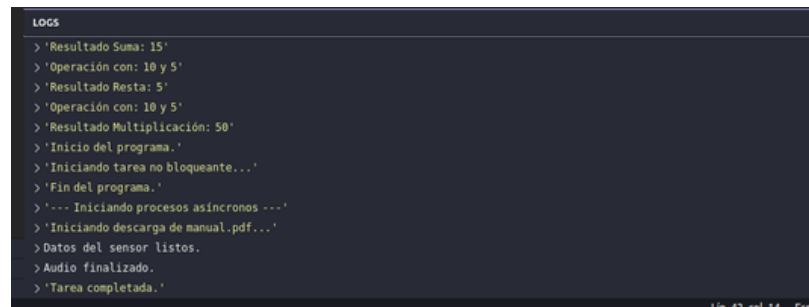
1. Mostrar un mensaje indicando que inició la descarga.
2. Esperar 4 segundos utilizando `setTimeout()`.
3. Mostrar un mensaje indicando que la descarga terminó.
4. Ejecutar el callback recibido.

Ejemplo esperado

```
simularDescarga("manual.pdf", () => {  
    console.log("Archivo listo para abrir.");  
});
```

Evidencia

Captura la salida de consola y explica qué sucede en cada paso.



Parte 5. Análisis del Callback Hell

Observa la siguiente estructura:

```
obtenerUsuario(() => {  
    obtenerPedidos(() => {  
        obtenerFactura(() => {  
            enviarCorreo(() => {  
                console.log("Proceso finalizado");  
            });  
        });  
    });  
});
```

Actividades

1. Describe el problema visual que presenta el código.
RTA/ El código tiene muchas funciones anidadas unas dentro de otras, formando una estructura difícil de leer conocida como "pirámide de la perdición" o "Pyramid of Doom".
2. Explica qué es el Callback Hell.
RTA/ Es una situación en la que varios callbacks se anidan excesivamente, haciendo que el código sea complicado de entender, depurar y mantener.
3. ¿Por qué dificulta el mantenimiento de aplicaciones grandes?
RTA/ Porque reduce la legibilidad del código, dificulta encontrar errores y hace más complejo agregar nuevas funcionalidades o modificar las existentes.
4. Investiga y menciona dos alternativas modernas para evitar este problema.
Promesas (Promises):
Permiten manejar operaciones asíncronas mediante `.then()` y `.catch()`, evitando la anidación excesiva.
Async/Await:
Permite escribir código asíncrono con una sintaxis similar al código secuencial, haciéndolo más claro y fácil de mantener.

Entregables

El estudiante deberá entregar:

- Código fuente de todos los ejercicios.
- Capturas de consola de las ejecuciones.
- Respuestas a las preguntas de análisis.
- Explicación del ejercicio de Callback Hell.